EGR111 – Introduction to Computer Science (C Language)

Visual Studio Code Introduction Rev 1.0

Overview:

In the Introduction to C Programming Introduction activity, you had a first look at the Visual Studio Code editor when you used the Programiz Online C Compiler. In this lesson you'll launch the Visual Studio Code application on your RPi system. This tool was installed and configured for you by the instructor. The instructor will first discuss the installation including required components and system settings. You'll then launch the tool, review the interface, write some initial code, and run. VS Code is the tool for this course. Practice with it to learn it well. As discussed in the earlier session, this is the tool of professional developers.

Prerequisites:

Prior to beginning the instruction provided in this lesson you must have completed the following:

1. Linux Introduction

Performance Outcomes:

1. Recognize the Visual Studio Code interface.
2. Identify requirements for VS Code to compile and execute C code.
3. Use the terminal to open a directory as a workspace in VS Code.
4. Use the VS Code Explore to create and access files in the workspace.
5. Use the VS Code editor to write and debug C code.
6. Use the Extensions collection in the Activity bar to view installed extensions.

Resources:

1. Getting started with Visual Studio Code
2. Download Visual Studio Code - Mac, Linux, Windows
3. How to fix slow Visual Studio Code on Raspberry Pi 4 - RATTICON

Materials:

1. RPi System

Directions:

1. Use the terminal commands learned in the Linux Introduction activity to create a workspace for this course. Open the terminal window and using the **mkdir** command to create an **EGR111** directory. You will use this workspace on the RPi system to complete the coding requirements for this class.

© 2022, Keith E. Kelly. All rights reserved.

2. Move to the EGR111 directory. Create a directory under this main directory named **Week01** Create a directory under this main directory named **Week02.** List the content of the EGR111 workspace. Your terminal window should look similar to the image below.

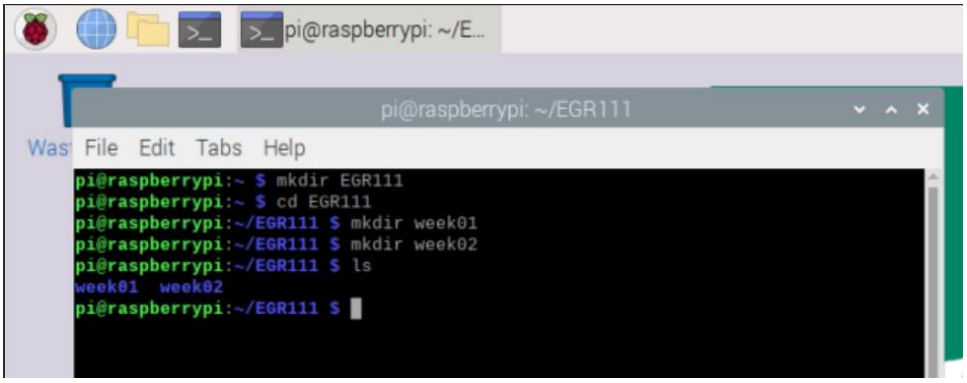

Figure 1 Terminal Window Showing Class Workspace Created

3. Open VS Code in your EGR111 workspace by entering the command ***code .*** with the dot after code indicating to open VS Code in the current directory.

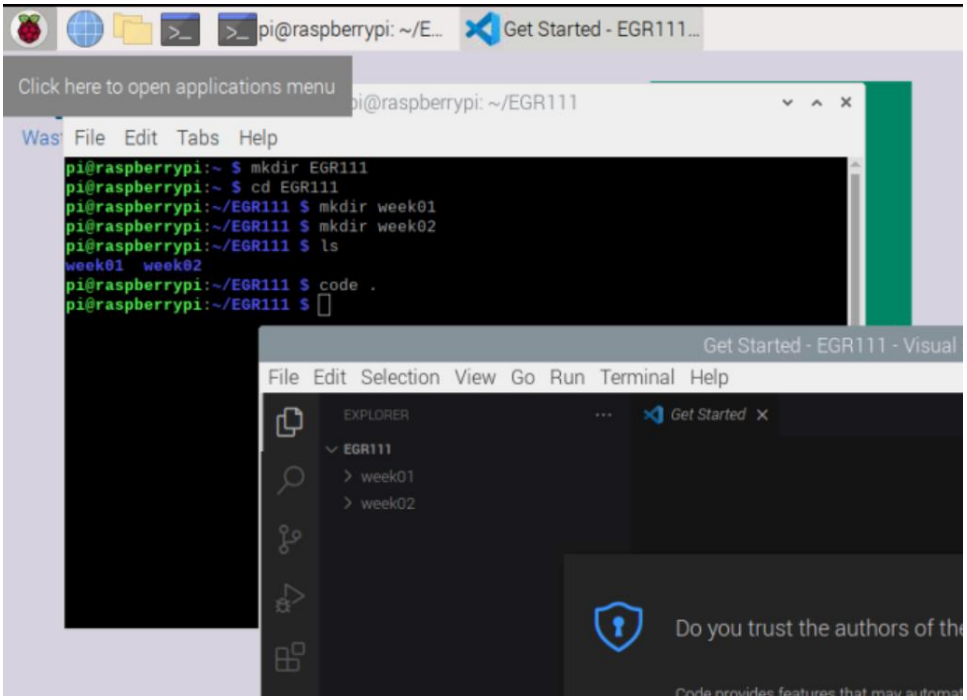

Figure 2 Visual Studio Code Opened on the Desktop using the code dot Command

4. Verify that your directory structure and the VS Code window matches what the instructor is demonstrating. You must manage your files and directories in this class so that the required structure is created.

© 2022, Keith E. Kelly. All rights reserved.

In future lessons you will be pushing your code to a repository that is shared with the instructor. Your files and structures must match those specified in lessons and assignments.

5. The instructor will provide an overview of the Visual Studio Code IDE. For additional details or to review, view the Getting started with Visual Studio Code page and associated video.
6. The instructor will discuss the requirements for creating and debugging C code using VS Code on the Raspberry Pi.
    a. A C compiler must be installed. The C compiler on your RPi unit is part of the Raspian operating system and can be viewed using the following commands.

```
cd /bin
gcc –version
```

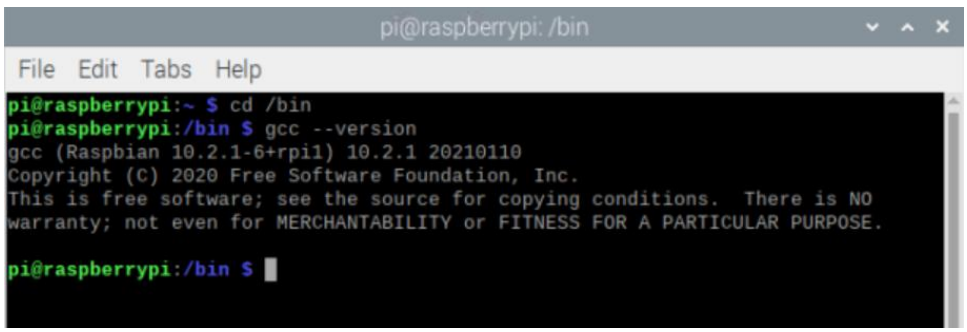

Figure 3 gcc compiler install in bin directory

    a. An editor or IDE is required. As discussed in a prior lesson, VS Code is the most popular IDE. This can be installed from Download Visual Studio Code - Mac, Linux, Windows but a Linux package is available. Enter the following command on a Linux command prompt will install VS Code.

```
sudo apt install code
```

    b. VS Code provides editing and debugging for almost any language. To use VS Code to create C solutions an extension must be installed. Note that other extensions related to C programming are recommended. These were not installed when the class version of the OS was created. The intent was to keep the editor as "light" as possible. We may experiment with other extensions as we complete the class.

© 2022, Keith E. Kelly. All rights reserved.

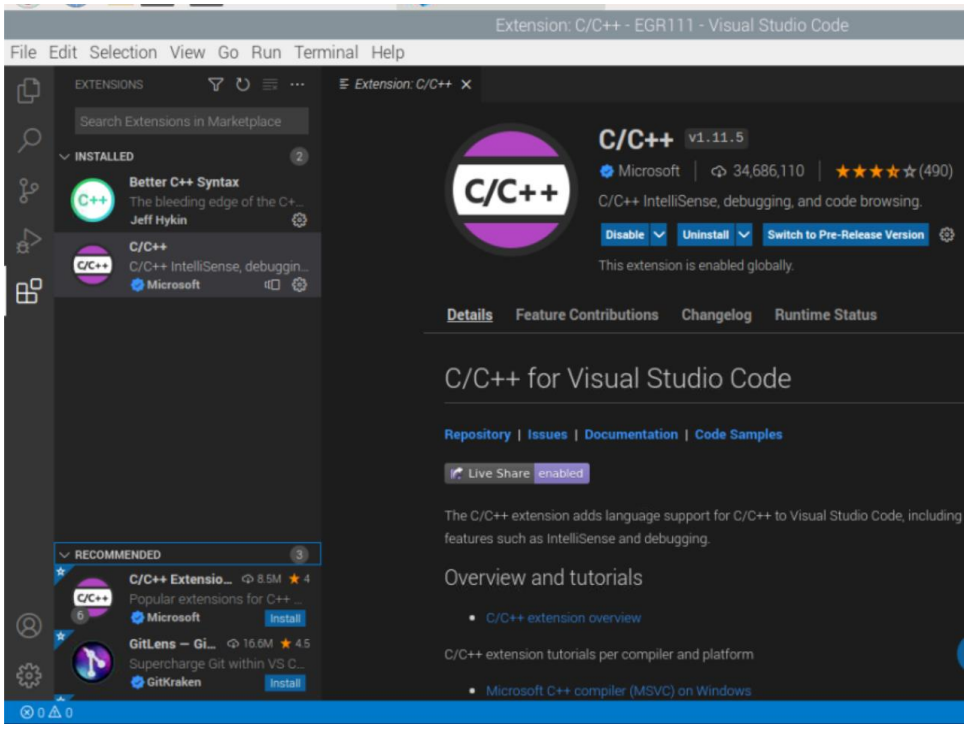

Figure 4 VS Code with C Extension Installed

7. Return to the Explorer tool on the Activity Bar and click on the week02 folder. We'll create our first C code – the perfunctory Hello World example.
8. Right-click on the week02 directory and select New File. Enter hello_world.c as the filename.

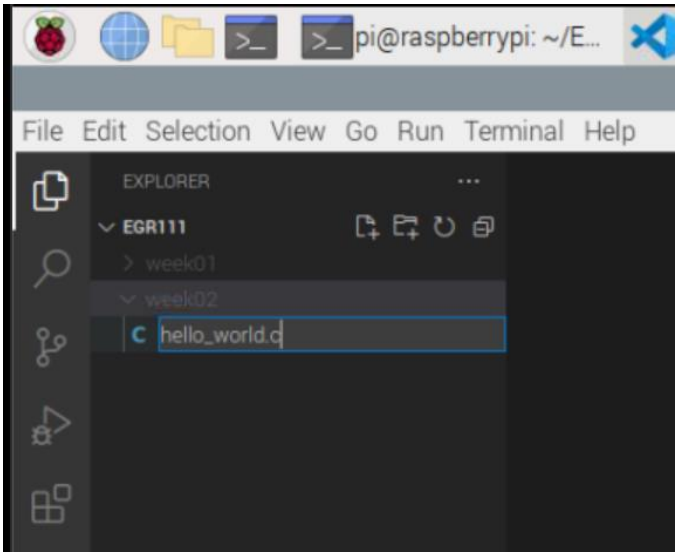

Figure 5 Creating the Hello World Example Code

© 2022, Keith E. Kelly. All rights reserved.

9. You'll be writing your own code soon enough. For now, you'll copy the example code provided by the instructor. The instructor will copy code to codeshare.io and then copy the URL to the Zoom chat window.

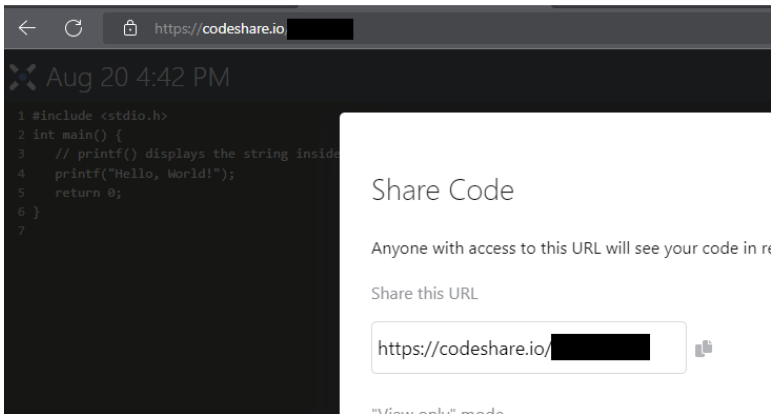

Figure 6 Code Pasted to CodeShare.io

10. Open the Zoom chat. Click on the link to open a copy of the code or enter the URL in the RPi browser. Press Ctrl-A to select all and then Ctrl-C to copy.

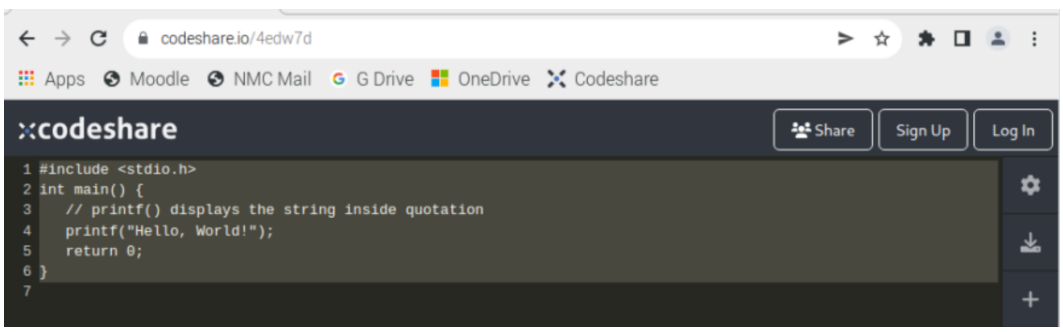

Figure 7 CodeShare Link Displaying Sample Code

11. Now return to the VS Code editor, click in the hello_world.c file and press Ctrl-C to copy.

Figure 8 Hello World Code Pasted into Editor

© 2022, Keith E. Kelly. All rights reserved.

12. Now let's run the code. Click Start Debugging from the Run menu or just press F5.

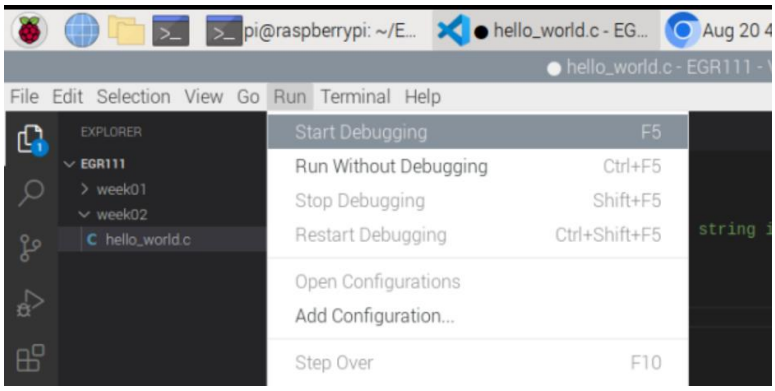

Figure 9 Running Code in VS Editor

13. You're prompted to have the installed compiler (gcc) build and debug the active file. Press Enter to select. The DEBUG CONSOLE shows the results of the build process.

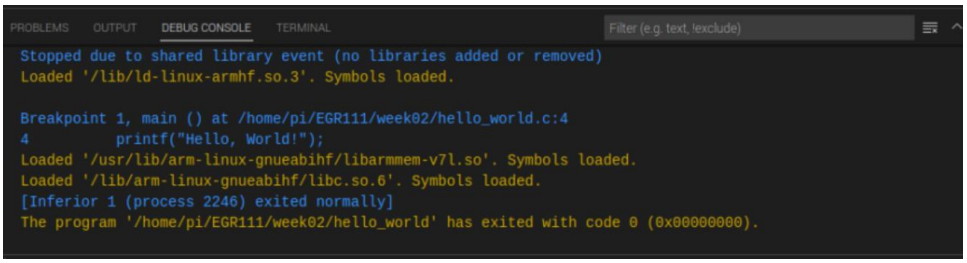

Figure 10 Debug Console in VS Code

14. Click on the TERMINAL window to see the resulting output.

Figure 11 Integrated Terminal Showing Result

15. Create a screen capture of your VS Code editor with the Hello World example code loaded. Name this file **hello_world.jpg** and submit to the Moodle link provided. In the next lesson, you'll learn to submit your code to a remote repository. Moodle assignment links will no longer be required.

© 2022, Keith E. Kelly. All rights reserved.

Assessments:

Submit your hello_world.jpg image to the Moodle link provided.

© 2022, Keith E. Kelly. All rights reserved.